

同一批请求，顺序会改变利用率

周六实验 · 2026-09-12 · 32 NPU / 3、4、6 SSU

最值得记住的实测结果

4 盘、同一 2-20 秒窗口：Random **98.90%**，Ordered **71.34%**，相差 27.56 个百分点。

输入直接取自 data；K=1024，miss 是新增计算的 token 数。

画像	总输入	miss	每层读 MiB	每层算 ms	V/C GiB/s
A · 大读取·短计算	128K	256	175.66	6.024	28.48
B · 小读取·长计算	32K	4096	38.50	28.593	1.31

每卡固定 **40A + 80B**，每请求 8 层、batch=1。32 卡共 3840 条请求，全部在 t=0 到达；每盘 40 GiB/s，每卡接收链路 50 GiB/s。

Random

每张卡独立打乱自己的同一批请求。

Ordered

每张卡都按 A→B→B 重复，起点相同；没有运行时同步屏障。

长验证窗口统一为 [2,20) 秒，T=18 秒；以下全部是 Baseline。

SSU 数	总容量 GiB/s	Random 平均利用率	Ordered 平均利用率
3	120	91.41%	64.22%
4	160	98.90%	71.34%
6	240	99.98%	97.24%

Random 是 seed 7、19、43 的算术平均；Ordered 是 seed 7。每格长窗内所有卡始终有请求，且每卡都计算过 A、B；并非每个更短子窗都逐卡包含两类。

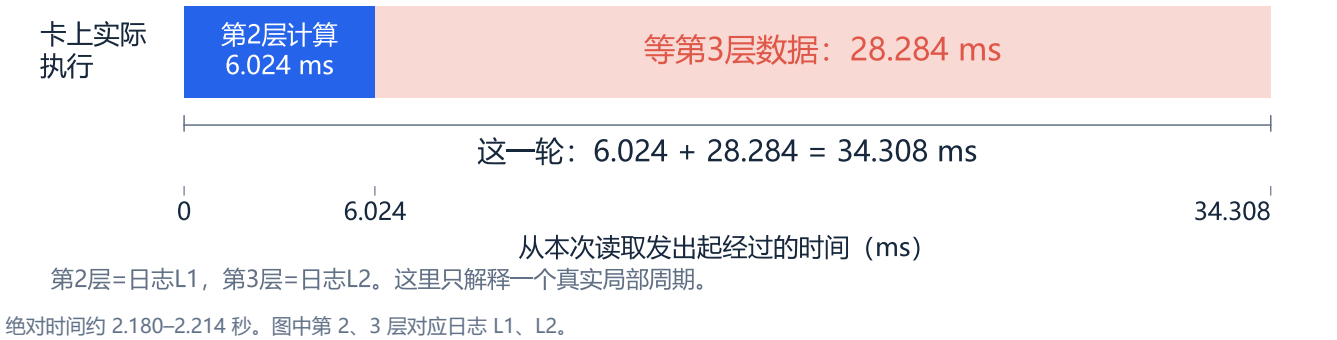
4 盘的低利用率延续到长窗；6 盘会恢复。6 盘 Ordered 从 2-4 秒的 86.59%，逐步恢复到后期约 100%，不能把最初低点当成持续结论。

直接原因：下一层数据没赶上计算

真实逐层时序 · 4 盘 Ordered / seed 7 · 不是教学拼接图

当前层一开始计算，就预取下一层。只有下一层数据到达 NPU 后，才能继续计算。下面是 NPU 0 的一个真实 A 请求片段。

4盘 Ordered · NPU 0 · A请求 9
真实绝对时间：2180.093–2214.401 ms



这一轮发生了什么	实际时间
当前层的纯计算	6.024 ms
下一层从读取发出到 NPU 收齐	34.308 ms
暴露的 IO 等待	28.284 ms

该局部周期利用率 = 6.024 / 34.308 = 17.56%

17.56% 只对应这一个约 34.3 ms 的局部周期，不是第 1 页的长窗利用率。该小段 32 卡均在 A，4 块盘都忙；整段算出的每卡平均服务速率为 5 GiB/s。

为什么 A 更容易等？

A 的内部层预取，要在前层约 6 ms 的计算期间读回 175.66 MiB。很多卡一起进入 A，读取集中到各盘 FIFO，下一层便可能来不及到齐。B 每层算得更久、读得更少，内部层的等待通常能被计算覆盖。

A→B 首层是例外：B 的首层在前一个 A 的末层计算时预取，预算只有 A 的 6.024 ms。日志中一次读取历时 31.629 ms，暴露等待 **25.605 ms**。

B 自己后续层的 28.593 ms 计算，不能倒过来替首层提供预取时间。其他卡的长计算本身也不占用本卡；共享争用发生在读取服务上。

[S2] formula_review/handoff_checks.json; local_A_cycle.svg。读取历时含排队、服务和到 NPU 的传输。

只记三个公式，分清两种时间

全部用纯文本；不需要 μ 、积分或长推导

1 • 实验报告使用：真实窗口平均

$$U_{\text{window}} = \text{窗口内所有卡累计的计算时间} / (N * T)$$

$N=32$ 。 t 是某一时刻，例如 3.2 秒； T 是窗口长度：2–4 秒的 $T=2$ 秒，2–20 秒的 $T=18$ 秒。跨边界的计算，只计入窗口内部分。

若只看某个 t ，真实瞬时利用率就是“此时正在计算的卡数 / N ”。窗口平均则累计整段时间；不能用不同结束时刻作分母，冒充同一暖窗对照。

2 • 描述请求：参考带宽需求

$$B_i(t) = V_i(t) / C_i(t)$$

i 是 NPU 编号。 V 是当前画像完整的一层读取量； C 是一层纯计算时间，均不是剩余量。画像不变时， B 保持不变；切换请求时可以变化。换算带宽时，统一用 GiB 和秒。

请求 A 的参考需求约 **28.48 GiB/s**，请求 B 约 **1.31 GiB/s**。计算时间直接查 data；若写 $t_c=C/m$ ，它是每 token 的平均分摊时间，会随总长度和 miss 数变化。

3 • 理想模型使用：周期平均

$$\begin{aligned} U_{\text{cycle}} &= C / \max(C, V / b) \\ &= \min(1, b / B) \end{aligned}$$

这里 b 是画像固定时持续可获得的带宽；假设计算与下一层读取理想重叠，忽略启动和结尾。它估计一张卡持续运行的平均水平。

不要把瞬时读取速率直接代入，就当真实瞬时利用率。数据提前读完后，实际读取速率可以是 0，卡仍然能计算。混合请求的任意短窗，也不保证与模型相等。

第 2 页选定的完整局部周期，可以核对 $5 / 28.48 \approx 17.56\%$ ；这不允许把同一个估计延伸到整段混合窗口。所有比值先得到 0–1，再显示为百分比。

调度要管“分多少”，也要管“何时读”

周六的直接对照是请求顺序；IO 策略效果仍需同输入验证

层次	它决定什么	周六对照
L1	请求分给哪张 NPU	保留同一逐卡人口与落盘
L2	每张卡按什么顺序执行请求	Random 与 Ordered 的差别
L3	已提交的 IO 先服务谁	始终使用 Baseline：每盘 Path0 FIFO

预算与先后，是两个问题

分多少：在固定画像、单一带宽池、可自由分配且只求平均 U 的模型里，未达到 100% 时，小 B 的卡每增加一份带宽能支撑更多计算；达到 100% 后不再增加。C 已包含在 V/C 中，不要求计算时长相近。

何时读：下一笔 IO 还要看什么时候必须读齐。给小 B 的卡留足预算，不代表每笔都让它先读；高 B 的卡当前层更紧急时，可以先服务它。

本次请求清单预先固定，但后续层 IO 随计算推进才提交。因此，服务顺序会改变未来读取出现的时间；仅凭“外部输入固定”，不能判断利用率不受调度影响。

本次已经支持什么？

相同 A/B 请求，集中进入“大读取·短计算”阶段会暴露等待。4 盘 Ordered 在 2–20 秒内明显较差；6 盘后期读取逐渐错开、利用率恢复。

还不能推出什么？

按完整配额且持续计算估出的平均需求约 124.91 GiB/s，低于 4 盘的 160 GiB/s；但所有长验证都曾有逐盘名义需求超限，**不是严格逐盘逐时刻欠载的反例**。本次 27 格全是 Baseline，**没有这批输入的 Once 对照**，也没有 A/B 不能互借带宽的配额。

整机 U 不能代替请求体验。4 盘 Ordered 长窗：A 类利用率仅 20.60%，B 类 97.23%（各类计算 / 该类占用卡时间）。评估策略还应看分类型结果和 TTFT/SLO；接纳后计时不等于外部到达后的端到端 TTFT。

原始来源

[S1] [周六实验报告与输入、结果 · 三种子汇总](#)

[S2] [公式、局部时序与适用条件核验](#)

完整研究：results/baseline_ab128_32_ratio12_20260912。本文只整理已有证据，没有启动新实验。